Deep Patel

Boston, MA | 646-961-2475 | deep.patel.0603@gmail.com | linkedin.com/in/dpatel06 | github.com/DPatel03 | Portfolio

Education

Boston University — Boston, MA
M.S. Data Science — Data Engineering at Scale, ML for Business Analytics, AI Ethics — *Expected May 2027*

CUNY Queens College — Queens, NY
B.A. Computer Science — Data Structures, Database Systems, OS, Software Engineering — *Dec 2024*

Technical Skills

Languages: Python, TypeScript, JavaScript, Java, SQL, C++
Frameworks & Libraries: Spring Boot, Flask, React, Node.js, Express, Pandas, PySpark, Streamlit, Leaflet
APIs & Security: REST APIs, JWT, OpenAPI/Swagger, Postman
AI & LLM: RAG pipelines, Google Gemini API, LLM prompt engineering, anti-hallucination pipeline design
Testing & CI/CD: PyTest, Jest, GitHub Actions, Docker, Git
Cloud & Infra: AWS (EC2, S3), Docker, Kubernetes, OpenShift, Linux
Databases: PostgreSQL, MongoDB, MySQL, SQLite | Formats: Parquet, GeoJSON

Experience

Prama — Remote
Software Engineer — *Jan 2025 – Aug 2025*

- Engineered a role-based access control layer using Spring Boot, decoupling authorization from business services and eliminating a class of permission bugs that had required repeated hotfixes.
- Cut deployment overhead by **40%** by standardizing EC2 release workflows with Docker and scripted provisioning, eliminating all manual environment setup steps.
- Reduced read latency by **35%** on critical MongoDB and PostgreSQL queries via compound indexes and application-level caching, validated under load testing.
- Traced recurring API failures under load to unhandled edge cases in token refresh logic; resolved with retry-backoff fix that eliminated the failure pattern entirely.

Tech Incubator at CUNY — New York, NY
Software Engineering Intern — *Feb 2024 – Apr 2024*

- Built destination search and booking modules for a full-stack travel app (Node.js/React), replacing static content with parameterized API calls and cutting average page load by **20%**.
- Refactored backend route handlers to consolidate duplicated middleware, reducing the Express routing layer by **30%** and compressing integration test runtime.
- Standardized GitHub branching conventions across a 6-person intern team, reducing merge conflicts and improving sprint velocity.

Projects

Gateway Cities Dashboard | *React/Vite, Flask, Python, Pandas, Parquet, Leaflet, Census API*

- Chose Parquet over raw CSV for the data layer — columnar storage enabled **5–8x faster** filter queries on ACS demographic slices without loading full datasets into memory.
- Built a Python ETL pipeline ingesting raw ACS 5-year CSVs, computing derived features (foreign-born %, education, homeownership, poverty rate, median income), and outputting typed Parquet datasets.
- Designed a Flask REST API with `lru_cache`-backed Parquet loads and parameterized city/metric endpoints; implemented Leaflet choropleth over Census TIGERweb GeoJSON with click-to-filter city selection.

Justice Lens | *Python, Streamlit, Pandas, Plotly, pydeck, Boston PD Open Data*

- Resolved **40 non-canonical allegation label variants** via a canonical mapping layer, enabling reliable trend analysis across IAD complaint data from 2011–2020.
- Engineered a multi-page Streamlit app with `@st.cache_data` on all heavy transforms, keeping page transitions **under 1s** on 100K+ row inputs.
- Linked officer incident records to IAD complaint histories via fuzzy name normalization across datasets with no shared key, surfacing repeat-complaint patterns.

Grant Cancellation Impact Analysis | *Python, Pandas, PostgreSQL, ACS Census API | Team*

- Designed forensic analytics framework analyzing 133K+ federal grant transactions; findings delivered directly to a U.S. Senator's office.
- Linked disbursement data with ACS demographics — uncovered high-diversity communities absorbed up to **175x higher per-capita cuts** than low-diversity counterparts.

Credit Risk Prediction & Decisioning System | *Python, PostgreSQL, scikit-learn, HMDA Data*

- Cleaned 200K+ HMDA loan records, engineered features, applied Platt scaling to calibrate risk scores, and implemented approve/review/reject decision logic evaluated with Brier score and calibration curves; outputs stored in PostgreSQL for full auditability.